

Joins in SQL Server with Examples

In this article, I am going to give an overview of **Joins in SQL Server with Examples**. As part of this article, we are going to discuss the following pointers related to SQL Server Joins.

1. **What are the different types of joins available in SQL Server?**
2. **What are ANSI and NON-ANSI Joins in SQL Server?**
3. **Inner Join in SQL Server?**
4. **What is Outer Join in SQL Server?**
5. **What is CROSS join in SQL Server?**
6. **Self-Join in SQL Server?**
7. **What is the difference between JOIN and UNION in SQL Server?**

The SQL Server Joins are used to retrieve the data from two or more related tables. In general, tables are related to each other using the primary key and foreign key relationship but it is not mandatory. The tables involved in the joins must have a common field. And based on that common field the SQL Server JOINS retrieves the records.

What are the different types of joins available in SQL Server?

The SQL Server Joins are classified into two types such as

1. ANSI format JOINS
2. NON-ANSI format JOINS

Again the ANSI format joins classified into three types such as

1. Inner join
2. Outer join
3. Cross join

Further, the outer join is divided into three types are as follows

1. Left outer join
2. Right outer join
3. Full outer join

NON-ANSI join in SQL Server are classified into four types such as

1. EQUI join
2. NON-EQUI join
3. SELF-join
4. Natural Join

Examples to Understand Joins in SQL Server:

We are going to use the following Employee and Projects tables to understand the SQL Server Joins. The EmployeeId column in the Projects table is the foreign key referencing to the Id column of the Employee table. The Id column is the Primary key column in the Employee table.

Employee

Id	Name	Department	Salary	Gender	Age	City
1001	John Doe	IT	35000	Male	25	London
1002	Mary Smith	HR	45000	Female	27	London
1003	James Brown	Finance	50000	Male	28	London
1004	Mike Walker	Finance	50000	Male	28	London
1005	Linda Jones	HR	75000	Female	26	London
1006	Anurag Mohanty	IT	35000	Male	25	Mumbai
1007	Priyanla Dewangan	HR	45000	Female	27	Mumbai
1008	Sambit Mohanty	IT	50000	Male	28	Mumbai
1009	Pranaya Kumar	IT	50000	Male	28	Mumbai
1010	Hina Sharma	HR	75000	Female	26	Mumbai

Projects

ProjectId	Title	ClientId	EmployeeId	StartDate	EndDate
1	Develop ecommerce website from scratch	1	1003	2021-08-04 14:33:38.080	2021-09-08 14:33:38.080
2	WordPress website for our company	1	1002	2021-08-04 14:33:38.080	2021-09-18 14:33:38.080
3	Manage our company servers	2	1007	2021-08-04 14:33:38.080	2021-09-28 14:33:38.080
4	Hosting account is not working	3	1009	2021-08-04 14:33:38.080	2021-10-08 14:33:38.080
5	MySQL database from my desktop application	4	1010	2021-08-04 14:33:38.080	2021-10-18 14:33:38.080
6	Develop new WordPress plugin for my business we...	2	NULL	2021-08-04 14:33:38.080	2021-10-28 14:33:38.080
7	Migrate web application and database to new server	2	NULL	2021-08-04 14:33:38.080	2021-11-07 14:33:38.080
8	Android Application development	4	1004	2021-08-04 14:33:38.080	2021-10-03 14:33:38.080
9	Hosting account is not working	3	1001	2021-08-04 14:33:38.080	2021-10-13 14:33:38.080
10	MySQL database from my desktop application	4	1008	2021-08-04 14:33:38.080	2021-10-23 14:33:38.080
11	Develop new WordPress plugin for my business we...	2	NULL	2021-08-04 14:33:38.080	2021-10-23 14:33:38.080

Please use the following SQL Script to create the Company database, Employee, and Projects tables with the required sample data.

```
-- Create Company Database
CREATE DATABASE Company;
-- Create Employee Table
CREATE TABLE Employee (
    Id INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Department VARCHAR(100) NOT NULL,
    Salary FLOAT NOT NULL,
    Gender VARCHAR(45) NOT NULL,
    Age INT NOT NULL,
    City VARCHAR(45) NOT NULL
);
-- Populate Employee Table
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'John Doe', 'IT', 35000, 'Male', 25, 'London');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Mary Smith', 'HR', 45000, 'Female', 27, 'London');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'James Brown', 'Finance', 50000, 'Male', 28, 'London');
```

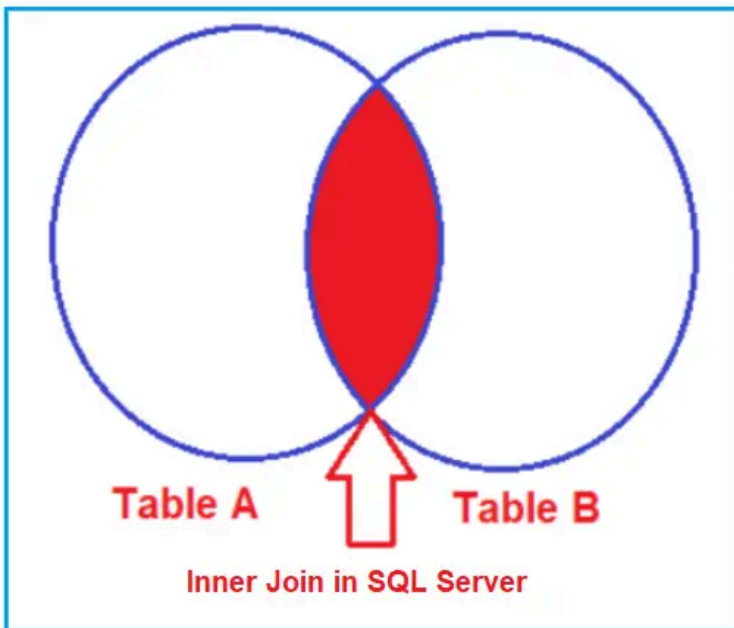
```

INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Mike Walker', 'Finance', 50000, 'Male', 28, 'London');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Linda Jones', 'HR', 75000, 'Female', 26, 'London');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Anurag Mohanty', 'IT', 35000, 'Male', 25, 'Mumbai');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Priyanla Dewangan', 'HR', 45000, 'Female', 27, 'Mumbai');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Sambit Mohanty', 'IT', 50000, 'Male', 28, 'Mumbai');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Pranaya Kumar', 'IT', 50000, 'Male', 28, 'Mumbai');
INSERT INTO Employee (Id, Name, Department, Salary, Gender, Age, City) VALUES
'Hina Sharma', 'HR', 75000, 'Female', 26, 'Mumbai');
-- Create Projects Table
CREATE TABLE Projects (
    ProjectId INT PRIMARY KEY IDENTITY(1, 1),
    Title VARCHAR(200) NOT NULL,
    ClientId INT,
    EmployeeId INT,
    StartDate DATETIME,
    EndDate DATETIME,
    FOREIGN KEY (EmployeeId) REFERENCES Employee(Id)
);
-- Populate Projects Table
INSERT INTO Projects (Title, ClientId, EmployeeId, StartDate, EndDate) VALUES
('Develop ecommerce website from scratch', 1, 1003, GETDATE(), (Getdate() + 35)),
('WordPress website for our company', 1, 1002, GETDATE(), (Getdate() + 45)),
('Manage our company servers', 2, 1007, GETDATE(), (Getdate() + 55)),
('Hosting account is not working', 3, 1009, GETDATE(), (Getdate() + 65)),
('MySQL database from my desktop application', 4, 1010, GETDATE(), (Getdate() + 75)),
('Develop new WordPress plugin for my business website', 2, NULL, GETDATE(), (Getdate() + 85)),
('Migrate web application and database to new server', 2, NULL, GETDATE(), (Getdate() + 95)),
('Android Application development', 4, 1004, GETDATE(), (Getdate() + 60)),
('Hosting account is not working', 3, 1001, GETDATE(), (Getdate() + 70)),
('MySQL database from my desktop application', 4, 1008, GETDATE(), (Getdate() + 80)),
('Develop new WordPress plugin for my business website', 2, NULL, GETDATE(), (Getdate() + 90));

```

Inner Join in SQL Server

The Inner Join in SQL Server is used to return only the matching rows from both the tables involved in the join by removing the non-matching records. The following diagram shows the pictorial representation of SQL Server Inner Join.



Syntax to use Inner Join in SQL Server:

Please have a look at the following image which shows the syntax of Inner Join in the SQL Server database. In SQL Server, you can use either the INNER JOIN or JOIN keyword to perform Inner Join. If you use only the JOIN keyword, then it is also going to perform Inner JOIN in SQL Server.

```
SELECT          column_name(s)
FROM            table1
INNER JOIN / JOIN table2
ON table1.column_name = table2.column_name;
```

Inner Join Example in SQL Server:

Our requirement is to retrieve the EmployeeId, Name, Department, City, Title as Project, and ClientId from the Employee and Projects tables. So, basically, we want the following result set as our output.

EmployeeId	Name	Department	City	Project	ClientId
1003	James Brown	Finance	London	Develop ecommerce website from scratch	1
1002	Mary Smith	HR	London	WordPress website for our company	1
1007	Priyanla Dewangan	HR	Mumbai	Manage our company servers	2
1009	Pranaya Kumar	IT	Mumbai	Hosting account is not working	3
1010	Hina Sharma	HR	Mumbai	MySQL database from my desktop application	4
1004	Mike Walker	Finance	London	Android Application development	4
1001	John Doe	IT	London	Hosting account is not working	3
1008	Sambit Mohanty	IT	Mumbai	MySQL database from my desktop application	4

Following is the SQL Server Inner Join Query that is joining the Employee and Projects tables and gives you the result set as shown in the above image. Here, we are using the EmployeeId column to check the similar values on the ON clause as both the tables having this column (Id in Employee table and EmployeeId in Projects table).

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
INNER JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

If you look at the output, here, we got 8 rows. We did not get those 3 records that have the NULL value in the EmployeeId column of the Projects table. So, this proves that the Inner Join in SQL Server is used to return only the matching records from both the tables involved in the join. The non-matching records are simply ignored or eliminated.

Instead of using the INNER JOIN keyword, we can also use the JOIN keyword as shown in the below Query, and we will also get the same output as the previous example. This also proves that the default joins in SQL Server is Inner Join.

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

Outer Join in SQL Server

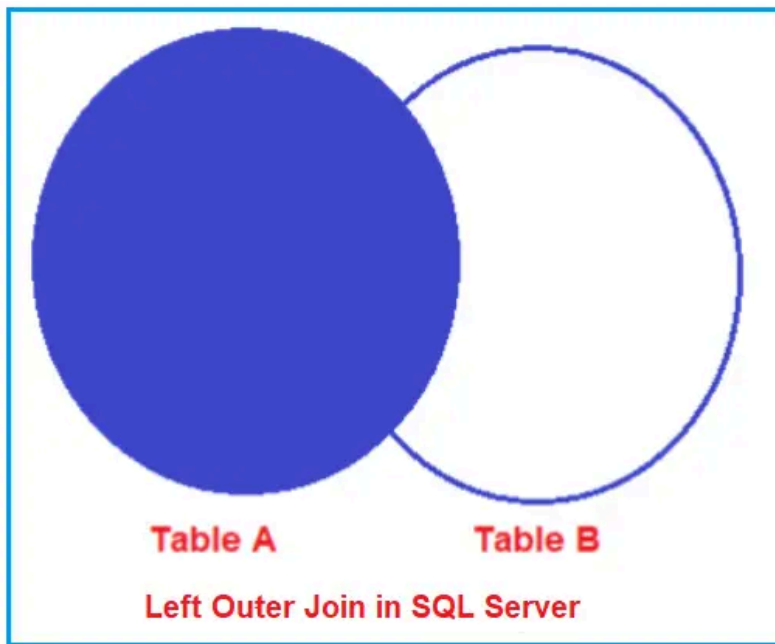
The OUTER JOIN in SQL Server returns matched data rows as well as unmatched data rows from both the tables involved in the join. Outer join in SQL Server is again classified into three types as follows.

1. **Left outer join**
2. **Right outer join**
3. **Full outer join**

Left Outer Join in SQL Server

The LEFT OUTER JOIN in SQL Server is used to retrieve all the matching rows from both the tables involved in the join as well as non-matching rows from the left side table. In this case, the un-matching data will take a null value.

The question that should come to your mind is, which is the left table and which is the right table? The answer is, the table mentioned to the left of the LEFT OUTER JOIN keyword is the left table, and the table mentioned to the right of the LEFT OUTER JOIN keyword is the right table. The following diagram is the pictorial representation of SQL Server Left Outer Join.



Syntax to use Left Outer Join in SQL Server:

Please have a look at the following image which shows the syntax of Left Outer Join in SQL Server. In SQL Server, you can use either the LEFT OUTER JOIN or LEFT JOIN keyword to perform Left Outer Join. If you only use the LEFT JOIN keyword, then it is also going to perform LEFT OUTER JOIN in SQL Server.

Syntax1:

```
SELECT column_name(s)
FROM table1
LEFT OUTER JOIN table2
ON table1.column_name = table2.column_name;
```

Syntax2:

```
SELECT column_name(s)
FROM table1
LEFT JOIN table2
ON table1.column_name = table2.column_name;
```

Example to Understand Left Outer Join in SQL Server

Now, we need to write a query to retrieve the Id as EmployeeId, Name, Department, City, and Title as Project from the Employee and Projects tables. So, basically, we want the query to produce the following result set as output. As you can see in the output, we are retrieving all the matching records from both the table as well as the non-matching records from the Employee table.

EmployeeID	Name	Department	City	Project	ClientId
1001	John Doe	IT	London	Hosting account is not working	3
1002	Mary Smith	HR	London	WordPress website for our company	1
1003	James Brown	Finance	London	Develop ecommerce website from scratch	1
1004	Mike Walker	Finance	London	Android Application development	4
1005	Linda Jones	HR	London	NULL	NULL
1006	Anurag Mohanty	IT	Mumbai	NULL	NULL
1007	Priyanla Dewangan	HR	Mumbai	Manage our company servers	2
1008	Sambit Mohanty	IT	Mumbai	MySQL database from my desktop application	4
1009	Pranaya Kumar	IT	Mumbai	Hosting account is not working	3
1010	Hina Sharma	HR	Mumbai	MySQL database from my desktop application	4

Now using SQL Server LEFT OUTER JOIN, we can join the Employee and Projects table and display the information of the employees and the projects they are working on, we also get the details of employees who are not working on any project and in that case, the Project and ClientId column will take NULL values. The Following Query will give you the results as shown in the above image.

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
LEFT OUTER JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

If you look at the output, here, we got all 10 records (i.e. all the records from the Employee Table) including the row that has a null value for the EmployeeId column in the Projects table. So, this proves that the Left Outer Join will retrieve all the rows from the Left-hand side Table including the rows that have a null foreign key value in the right-hand side table.

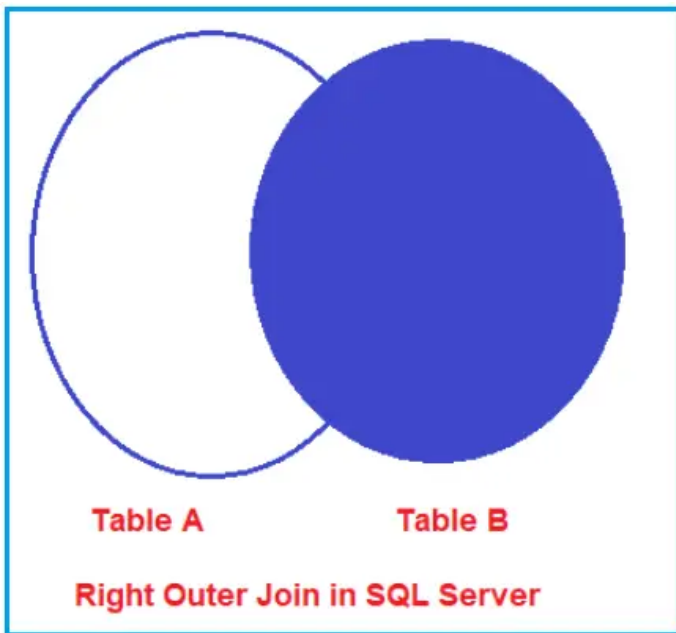
Instead of using the **LEFT OUTER JOIN** keyword, you can also use the **LEFT JOIN** keyword as shown in the below Query, and also you will get the same output as the previous query.

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
LEFT JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

Right Outer Join in SQL Server

The RIGHT OUTER JOIN in SQL Server is used to retrieve all the matching rows from both the tables involved in the join as well as non-matching rows from the right-side table. In this case, the un-matching data will take NULL values.

The question that should come to your mind is, which is the left table and which is the right table? The answer is, the table mentioned to the left of the RIGHT OUTER JOIN keyword is the left table, and the table mentioned to the right of the RIGHT OUTER JOIN keyword is the right table. The following diagram is the pictorial representation of SQL Server Right Outer Join.



Syntax to use Right Outer Join in SQL Server:

Please have a look at the following image which shows the syntax of Right Outer Join in SQL Server. In SQL Server, you can use either the RIGHT OUTER JOIN or RIGHT JOIN keyword to perform Right Outer Join. If you only use the RIGHT JOIN keyword, then it is also going to perform RIGHT OUTER JOIN in SQL Server.

Syntax1:

```
SELECT column_name(s)
FROM table1
RIGHT OUTER JOIN table2
ON table1.column_name = table2.column_name;
```

Syntax2:

```
SELECT column_name(s)
FROM table1
RIGHT JOIN table2
ON table1.column_name = table2.column_name;
```

Example to Understand Right Outer Join in SQL Server

Now, we need to write a query to retrieve the Id as EmployeeId, Name, Department, City, and Title as Project from the Employee and Projects tables. So, basically, we want the query to produce the following result set as output. As you can see in the output, we are retrieving all the matching records from both the table as well as the non-matching records from the Projects table.

EmployeeID	Name	Department	City	Project	ClientId
1003	James Brown	Finance	London	Develop ecommerce website from scratch	1
1002	Mary Smith	HR	London	WordPress website for our company	1
1007	Priyanla Dewangan	HR	Mumbai	Manage our company servers	2
1009	Pranaya Kumar	IT	Mumbai	Hosting account is not working	3
1010	Hina Sharma	HR	Mumbai	MySQL database from my desktop application	4
NULL	NULL	NULL	NULL	Develop new WordPress plugin for my business we...	2
NULL	NULL	NULL	NULL	Migrate web application and database to new server	2
1004	Mike Walker	Finance	London	Android Application development	4
1001	John Doe	IT	London	Hosting account is not working	3
1008	Sambit Mohanty	IT	Mumbai	MySQL database from my desktop application	4
NULL	NULL	NULL	NULL	Develop new WordPress plugin for my business we...	2

Now using SQL Server RIGHT OUTER JOIN, we can join Employee and Projects table and display the information of employees and the projects they are working on, we also get the details of projects which are not yet assigned to any employee. The Following Query will give you the results as shown in the above image.

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
RIGHT OUTER JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

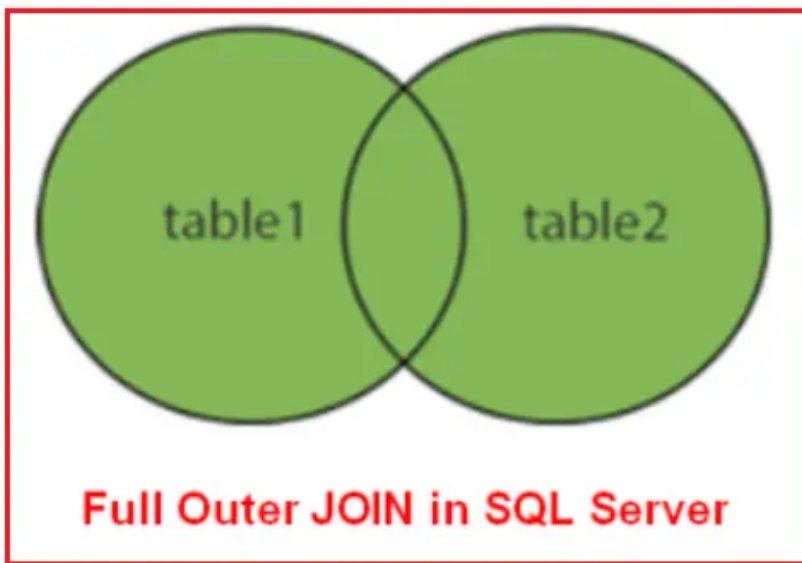
If you look at the output, here, we got all 11 rows (i.e. all the rows from the Projects Table). If you further notice, here, we got all the matching records from both the Employee and Projects tables as well as all non-matching rows from the right-side table i.e. the Projects Table. So, this proves that the Right Outer Join retrieves all the rows from the Left-hand side Table including the rows that have a null foreign key value in the left-hand side table.

Instead of using the **RIGHT OUTER JOIN** keyword, we can also use the **RIGHT JOIN** keyword as shown in the below SQL Query, and also get the same output as the previous query.

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
RIGHT JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

Full Outer Join in SQL Server

The Full Outer Join in SQL Server is used to retrieve all the matching records as well as all the non-matching records from both the tables involved in the JOIN. The Un-matching data in such cases will take the NULL values. The following diagram shows the pictorial representation of Full Outer Join in SQL Server.



Syntax to use Full Outer Join in SQL Server:

Please have a look at the following image which shows the syntax of Full Outer Join in SQL Server. In SQL Server, you can use either the FULL OUTER JOIN or FULL JOIN keyword to perform Full Outer Join. If you only use the FULL JOIN keyword, then it is also going to perform FULL OUTER JOIN in SQL Server.

```
SELECT column_name(s)
FROM table1
FULL OUTER JOIN table2
ON table1.column_name = table2.column_name
WHERE condition;
```

Example to Understand Full Outer Join in SQL Server

Now, we need to write a query to retrieve the Id as EmployeeId, Name, Department, City, and Title as Project from the Employee and Projects tables. So, basically, we want the query to produce the following output. As you can see in the output, we are retrieving all the matching records as well as all the non-matching records from both the table.

EmployeeID	Name	Department	City	Project	ClientId
1001	John Doe	IT	London	Hosting account is not working	3
1002	Mary Smith	HR	London	WordPress website for our company	1
1003	James Brown	Finance	London	Develop ecommerce website from scratch	1
1004	Mike Walker	Finance	London	Android Application development	4
1005	Linda Jones	HR	London	NULL	NULL
1006	Anurag Mohanty	IT	Mumbai	NULL	NULL
1007	Priyanka Dewangan	HR	Mumbai	Manage our company servers	2
1008	Sambit Mohanty	IT	Mumbai	MySQL database from my desktop application	4
1009	Pranaya Kumar	IT	Mumbai	Hosting account is not working	3
1010	Hina Sharma	HR	Mumbai	MySQL database from my desktop application	4
NULL	NULL	NULL	NULL	Develop new WordPress plugin for my business we...	2
NULL	NULL	NULL	NULL	Migrate web application and database to new server	2
NULL	NULL	NULL	NULL	Develop new WordPress plugin for my business we...	2

Now using SQL Server FULL OUTER JOIN, we can join Employee and Projects table and display the information of employees and the projects they are working on, we also get the details of projects which are not yet assigned to any employee as well as all the employees who are not assigned with any projects. The Following Query will give you the results as shown in the above image.

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
FULL OUTER JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

If you look at the output, here, we got all 13 rows (i.e. all the rows from the Projects Table as well as all the rows from the Employee table). Instead of using the **FULL OUTER JOIN** keyword, we can also use the **FULL JOIN** keyword as shown in the below Query, and get the same output as the previous query.

```
SELECT Id as EmployeeID, Name, Department, City, Title as Project, ClientId
FROM Employee
FULL JOIN Projects
ON Employee.Id = Projects.EmployeeId;
```

Cross Join in SQL Server

The CROSS JOIN is created by using the CROSS JOIN keyword. The CROSS JOIN does not contain an ON clause. In Cross Join, each record of a table is joined with each record of the other table. In SQL Server, the Cross Join should not have either an ON or where clause.

Example to Understand CROSS JOIN in SQL Server:

A Cross Join in SQL Server produces the Cartesian product of the tables involved in the join. The Cartesian product means the number of records present in the first table is multiplied by the number of records present in the second table. Please have a look at the below SQL query which is an example of Cross Join for joining the Employee and Projects Table.

```
SELECT Employee.Id as EmployeeId, Name, Department, City, Title as Project
FROM Employee
```

CROSS JOIN Projects;

The Employee is the LEFT Table which contains 10 rows and Projects is the RIGHT Table which contains 11 rows. So, when you execute the above query, you will get 110 records in the result set.

What is Self-Join in SQL Server?

Joining a table by itself is called self-join in SQL Server. When we have some relation between the columns within the same table then we need to use the self-join mechanism.

When we implement the self-join in SQL Server, we should create the alias for the table name. We can create any number of aliases for a single table. Self-join is not a different kind of join. It can be classified as any type of join, such as

1. Inner Join
2. Outer (Left, Right, Full) join
3. Cross Join

Note: The Self-join in SQL Server is just like any other joins except that the two instances of the same table will be joined in the query.

What is the difference between Union and Join in SQL Server?

Joins and Unions are two different things. The Union in SQL Server is used to combine the result-set of two or more select queries into a single result-set. On the other hand, the Joins in SQL Server are used to retrieve the data from two or more related tables involved in the join.

What is the General Formula for Joins in SQL Server?

```
SELECT ColumnList FROM LeftTableName  
JOIN_TYPE RightTableName  
ON JoinCondition
```

In the next article, I am going to discuss [Inner Join in SQL Server](#) with real-time examples. Here, in this article, I try to give an overview of **SQL Server Joins**. I hope you enjoy this article.